

Aufgabe 2.3

Wir betrachten den ursprünglich gegebenen Cache:

- **Adressbreite:** 8 Bit.
- **Daten:** Halfword (2 Bytes) pro Adresse.
- **Struktur:** Tag (3 Bit) | Index (3 Bit) | Offset (2 Bit).
- **Größe:** 8 Lines, Direct Mapped.

a) Hit oder Miss?

Wir zerlegen jede Adresse binär (Tag - Index - Offset):

- Tag: Bits 7-5
- Index: Bits 4-2
- Offset: Bits 1-0

Adresse	Binär (T-I-O)	Tag	Index	Hit/Miss	Aktion im Cache
0xDC	110 111 00	6	7	Miss	Setze Index 7 auf Tag 6
0x90	100 100 00	4	4	Miss	Setze Index 4 auf Tag 4
0xD3	110 100 11	6	4	Miss	Konflikt! Index 4: Tag 4 → 6
0x0C	000 000 11	0	3	Miss	Setze Index 3 auf Tag 0
0xDD	110 111 01	6	7	Hit	Index 7 enthält Tag 6
0x93	100 100 11	4	4	Miss	Konflikt! Index 4: Tag 6 → 4
0xD0	110 100 00	6	4	Miss	Konflikt! Index 4: Tag 4 → 6
0x0D	000 000 11	0	3	Hit	Index 3 enthält Tag 0

Berechnung der Hit-Rate:

- Anzahl Zugriffe: 8
- Anzahl Hits: 2 (0xDD, 0x0D)

$$\text{Hit-Rate} = \frac{2}{8} = 0.25 = 25\%$$

—

FRAGE: bei D3, 93 zwei Cache Lines wegen Alignment befüllt? Bei D3: D0 D1 D2 D3, Index 05: D4 D5 D6 D7

b) Abschließende Belegung des Caches

Nach der Ausführung aller Befehle sieht der Cache wie folgt aus:

- **Index 3:** Tag 0 (aus 0x0D/0x0C)
- **Index 4:** Tag 6 (aus 0xD0)
- **Index 7:** Tag 6 (aus 0xDC/0xDD)
- **Alle anderen Indizes:** Leer (Invalid)

c) Umwandlung in 2-fach assoziativen Cache

Annahme: Die Adressstruktur (Tag 3 Bit, Index 3 Bit, Offset 2 Bit) bleibt identisch. Das bedeutet, wir haben weiterhin $2^3 = 8$ Sets (Indizes). Da der Cache nun 2-fach assoziativ ist, gibt es pro Set 2 Speicherplätze ("Ways").

1. **Veränderung der Hit-Rate:** Wir analysieren erneut die Konflikte bei Index 4 (0x90, 0xD3, 0x93, 0xD0):

- 0x90 (Tag 4, Set 4): **Miss**. Set 4 speichert [Tag 4, -].
- 0xD3 (Tag 6, Set 4): **Miss**. Set 4 speichert [Tag 4, Tag 6]. (Kein Konflikt, da 2 Plätze!)
- 0x93 (Tag 4, Set 4): **Hit**. Tag 4 ist im Set enthalten.
- 0xD0 (Tag 6, Set 4): **Hit**. Tag 6 ist im Set enthalten.

Die Zugriffe 0x93 und 0xD0, die vorher Misses waren, sind nun Hits.

- Neue Hits: 0xDD, 0x93, 0xD0, 0xD0.
- Neue Anzahl Hits: 4.

$$\text{Neue Hit-Rate} = \frac{4}{8} = 50\%$$

2. **Veränderung der Größe:**

- Anzahl Sets: 8 (bestimmt durch 3 Bit Index).
- Blockgröße: 8 Bytes (bestimmt durch 2 Bit Offset, unverändert).
- Assoziativität: 2 Blöcke pro Set.

$$\text{Neue Größe} = 8 \text{ Sets} \times 2 \text{ Blöcke/Set} \times 8 \text{ Bytes} = 128 \text{ Bytes}$$

Antwort: Die Größe des Caches **verdoppelt sich** von 64 Bytes auf 128 Bytes.

d) Reduzierung von Cache-Misses

Es gibt grundsätzlich zwei Hauptstrategien, um Misses zu reduzieren, die auf unterschiedlichen Prinzipien basieren:

1. **Erhöhung der Blockgröße:**

- **Prinzip: Räumliche Lokalität (Spatial Locality).**
- Wenn auf ein Datum zugegriffen wird, ist es sehr wahrscheinlich, dass kurz darauf auf benachbarte Adressen zugegriffen wird. Größere Blöcke laden diese Nachbarn gleich mit in den Cache ("Prefetching"-Effekt), was Misses bei sequentiellen Zugriffen reduziert.

2. **Erhöhung der Assoziativität:**

- **Prinzip:** Reduzierung von **Conflict Misses**.
- In einem direkt abbildenden Cache können zwei Adressen, die zufällig den gleichen Index haben, sich ständig gegenseitig verdrängen ("Thrashing"), selbst wenn der Cache eigentlich noch leer wäre. Assoziativität erlaubt es, mehrere Blöcke mit gleichem Index gleichzeitig zu speichern.